Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 13 - PAO 2026 – II

Agenda

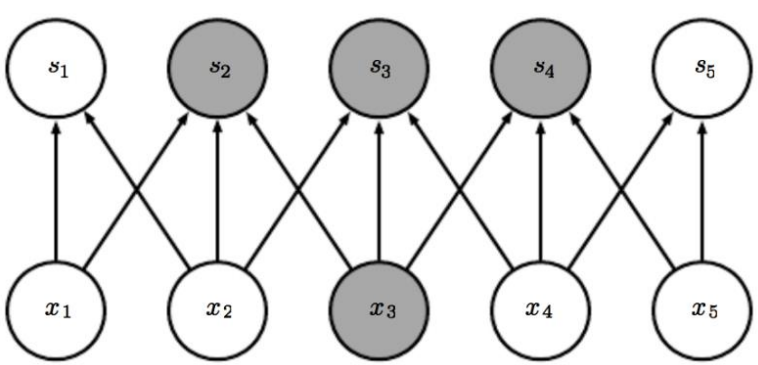
1. Redes neuronales convolucionales
   - 1.1 Traslational Equivariance y Shift Invatiance.
   - 1.2 Stride y Padding
   - 1.3 Capas Conv., volúmenes y cantidad de parámetros
   - 1.4 Capas Pooling
   - 1.5 Aprendizaje en las CNN: Número de parámetros a aprender

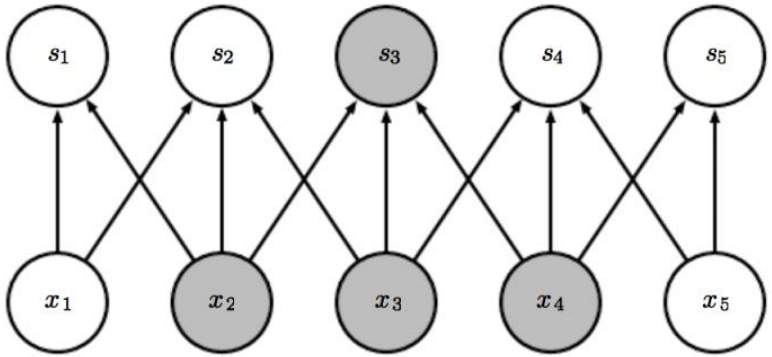
Redes neuronales convolucionales

- Equivarianza Translacional (Translational Equivariance y Shift invariance):
  - Equivarianza traslacional o invarianza traslacional, se refiere a la capacidad de compartir pesos, eso produce un efecto en el reconocimiento de patrones o características en una imagen independientemente de su posición en el espacio.
  - Esto se logra por la compartición de los pesos, si hay un objeto en cualquier imagen, se detectará independientemente de su posición en la imagen
  - Recordar: Shift invariance – preserva la estructura espacial, permitiendo a las CNN identificar características similares (como bordes, texturas o formas) en diferentes partes de una imagen sin importar su ubicación exacta.
  - Notar que las CNN no realizan transformaciones, como los cambios en la escala o la rotación de la imagen. Se requieren otros mecanismos para manejar tales transformaciones.

Redes neuronales convolucionales

- Conexiones dispersas en lugar de conexiones completamente conectadas

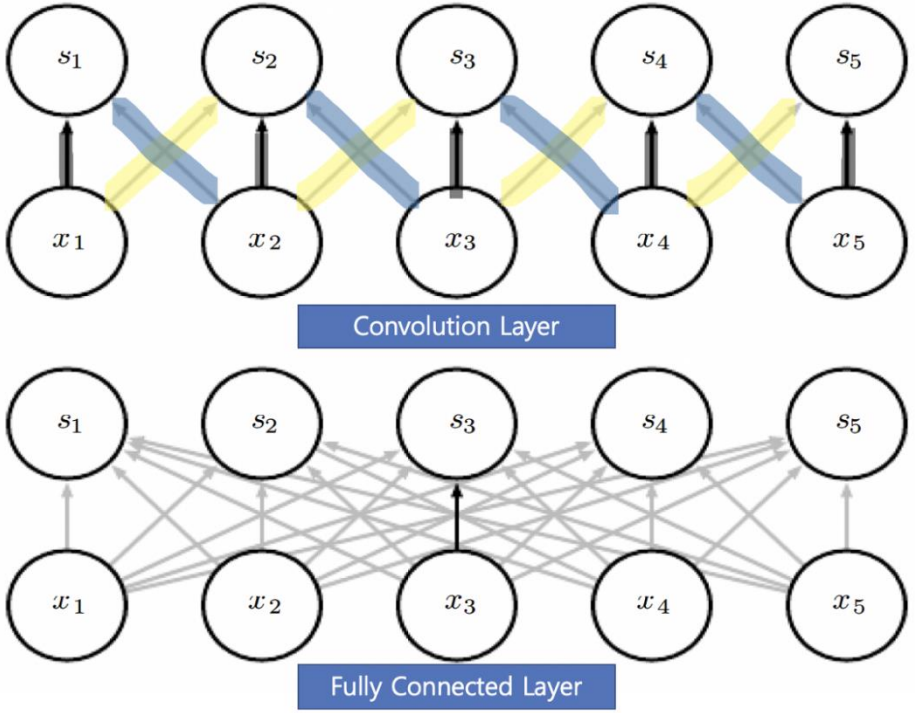




En las redes CNN las neuronas se conectan con algunas de las otras neuronas

Redes neuronales convolucionales

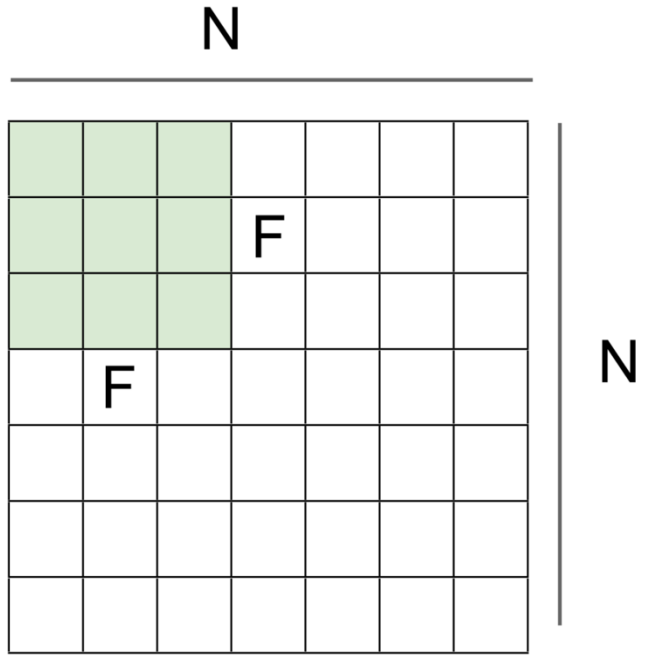
- Se comparten los pesos en toda la imagen



Redes neuronales convolucionales

- Stride (o paso) es el tamaño del paso que el filtro de convolución utiliza cada vez que se mueve
- Por ejemplo un paso de tamaño 1, significa que el filtro se desliza píxel por píxel.
- Al aumentar el tamaño del paso, el filtro se desliza sobre la entrada con un intervalo mayor y por lo tanto tiene menos superposición entre las celdas.

Redes neuronales convolucionales



Tamaño de la salida:

((N – F)/stride) + 1

Ej: . N = 7, F = 3:

stride 1 => (7 - 3)/1 + 1 = 5

stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33

Redes neuronales convolucionales

- Debido a que el tamaño del mapa de características siempre es más pequeño que la entrada, para evitar que el mapa se reduzca, usamos Padding (o relleno)
- El padding consiste en agregar una capa de píxeles de valor cero alrededor de la imagen, así el mapa de características no se reduce
- Además de mantener el tamaño, después de realizar la convolución, el padding también mejora el rendimiento y asegura que el tamaño del kernel y el stride encajen con la imagen

Redes neuronales convolucionales

Otro ejemplo:

- Imagen de entrada: 32x32x3
- 10 filtros 5x5 con stride 1, pad 2
- Cuál es el tamaño del mapa de salida:?

  (32+(2*2)-5)/1 + 1 = 32
- Cuál es el volumen de salida: ?

  32x32x10

Redes neuronales convolucionales

Y cuál es el número de parámetros ?

- Imagen de entrada: 32x32x3
- 10 filtros 5x5 con stride 1, pad 2
- Cuál es el número de parámetros en esta capa:?

  Cada filtro tiene: 5*5*3+1 = 76 parámetros (1 de bias)
- Entonces en total esta capa tiene:

  76*10 = 760 parámetros

Redes neuronales convolucionales

- Otro ejemplo: Supongamos que tenemos un volumen de entrada de 60x60x14, y se usan 25 filtros de 3x3, usando stride 1. Cuál es el padding para mantener el mismo tamaño del volumen ?

$((N+2P-F)/S)+1=60$

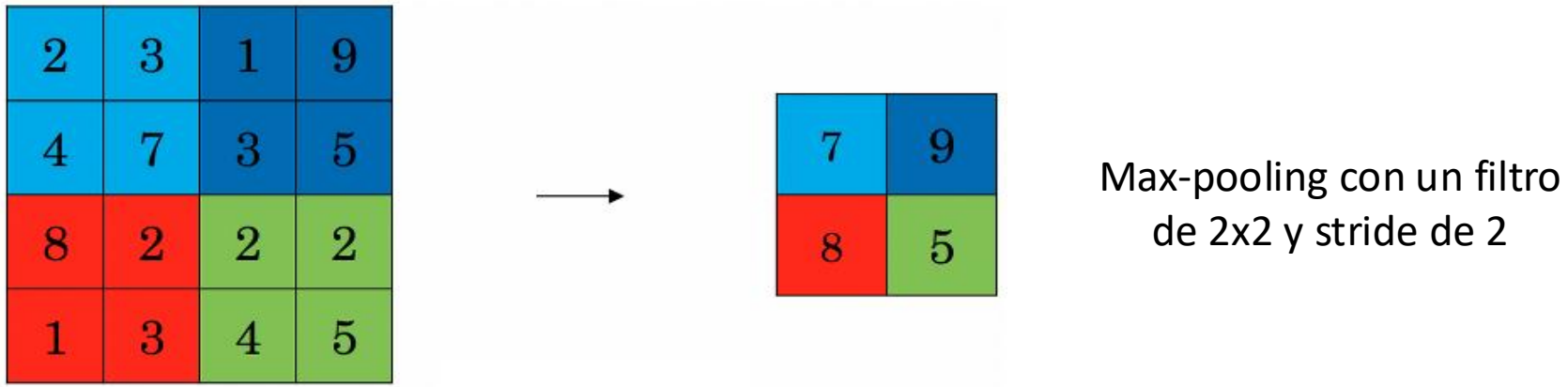
$P=(60S-N+F-S)/2$

$P=(60*1-60+3-1)/2$

$P=1$

Redes CNN: Extracción de características

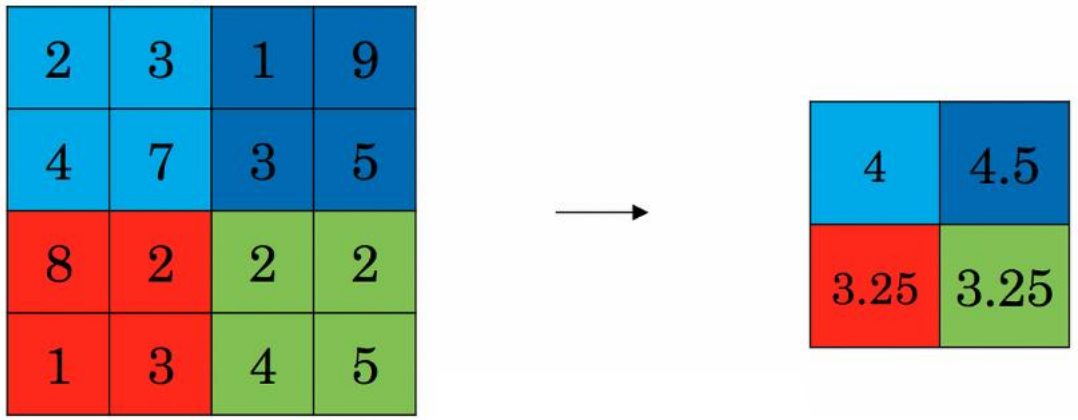
- Hay varios tipos de pooling, por ejemplo: Max Pooling, Pooling promedio, Global Pooling…
- Max pooling se basa en filtrar el valor máximo de la sección de la imagen cubierta por un filtro.
- Max Pooling también funciona como un supresor de ruido.
- Descarta la activación ruidosa junto con la reducción de dimensionalidad.



Max-pooling con un filtro de 2x2 y stride de 2

Redes CNN: Extracción de características

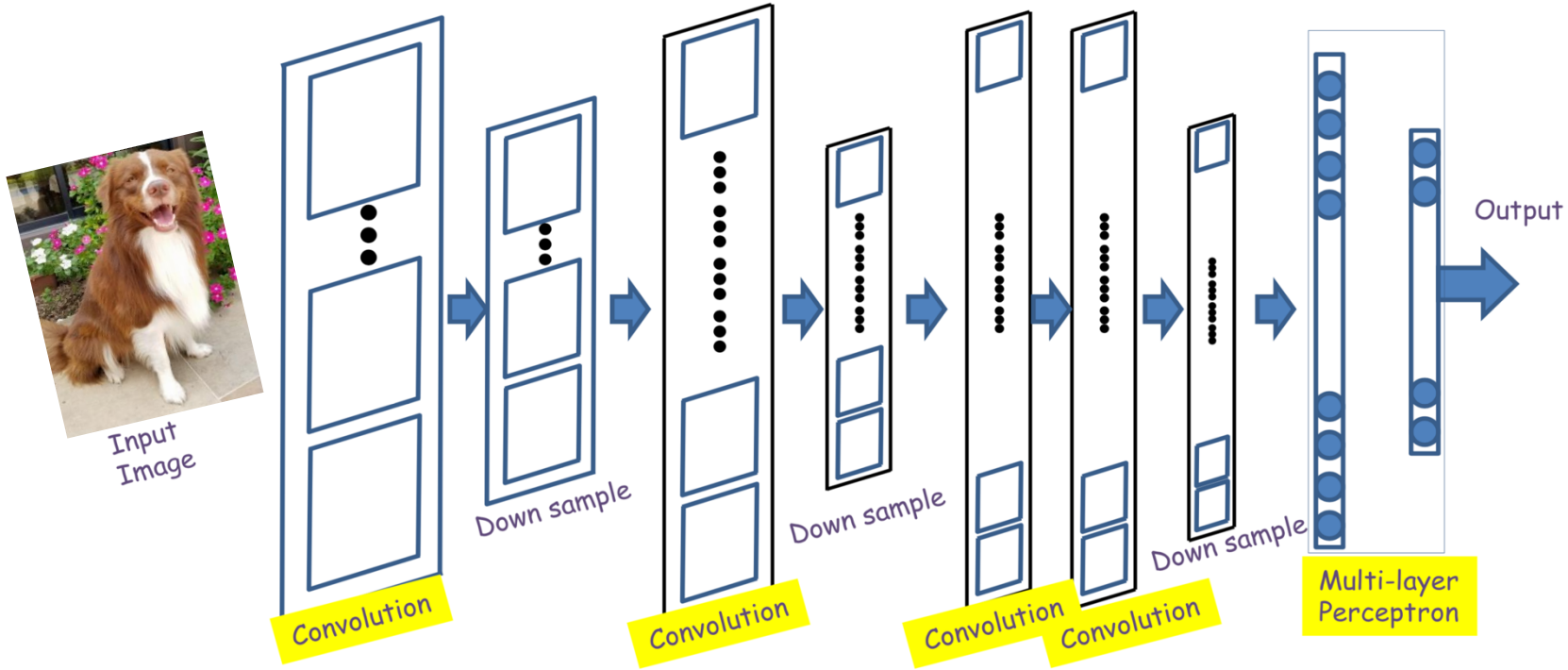
- Pooling promedio
  - El pooling promedio filtra el promedio de todos los valores de la parte de la imagen cubierta por el Kernel.
  - El pooling promedio también realiza la reducción de dimensionalidad como un mecanismo de supresión de ruido.



Pooling promedio con un filtro de 2x2 y stride de 2

Redes neuronales convolucionales

- Los parámetros de las capas convolucionales y la capa MLP se ”aprenden”
- Las capas de reducción (pooling) son fijas y no se aprenden



Redes CNN: Extracción de características

- Después de la convolución y las capas de pooling, la etapa de clasificación consta de pocas capas totalmente conectadas.
- Estas capas totalmente conectadas sólo pueden aceptar datos de 1 dimensión.
- Convertir los datos 3D a 1D.
- Por ejemplo, en Python usamos la función “flatten”.
- Las últimas capas de una CNN son capas totalmente conectadas.

Redes neuronales convolucionales

- Cada capa *j* tiene K_j mapas.
  - K_0 es el numero de “mapas” en la entrada (3 canales de colores)
- Con respecto a los parámetros en el resto de las capas:
  - Si L_j x L_j es el tamaño de los filtros en una capa J^{th}
  - Para esa capa J^{th} se requieren $K_j\left(K_{j-1}L_j^2 + 1\right)$ parámetros en los filtros
  - El total de parámetros requeridos para todas las capas convolucionales:

$$\sum_{J \in capas\ conv} K_j\left(K_{j-1}L_j^2 + 1\right)$$

Redes neuronales convolucionales

En nuestro ejemplo anterior:

- Imagen de entrada: 32x32x3
- 10 filtros 5x5 con stride 1, pad 2
- Esta capa tiene:

- $K_j\left(K_{j-1}L_j^2 + 1\right)$ = 10 * (3*5^2+1) = 760

Redes neuronales convolucionales

Híper-parámetros (opciones de diseño) que debemos escoger:

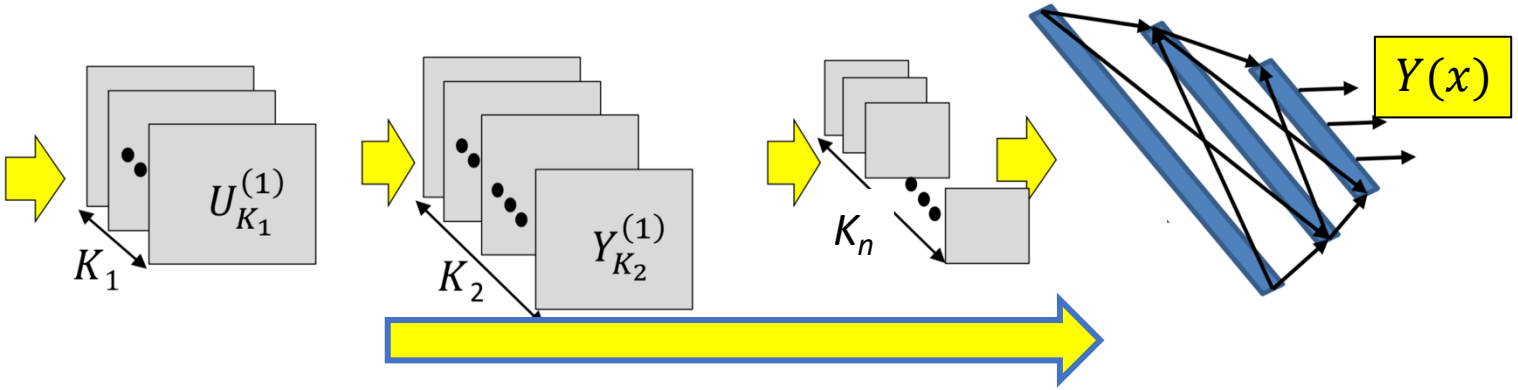
- Número de capas convolucionales, sus funciones de umbra y de reducción (o pooling)
  - El orden en el que se ubican
- Para cada capa de convolución:
  - Número de filtros K_i
  - Tamaño del filtro $L_i \times L_i$
  - La "profundidad" del filtro se fija por el número de filtros en la capa anterior K_{i-1}
- El stride S_i en las capas de convolución
- Para cada capa de reducción o pooling:
  - Tamaño espacial del filtro $P_i \times P_i$
  - El stride D_i
- Para el MLP final:
  - El número de capas y número de neuronas en cada capa y sus funciones umbral

Redes neuronales convolucionales

Aprendizaje en CNN:

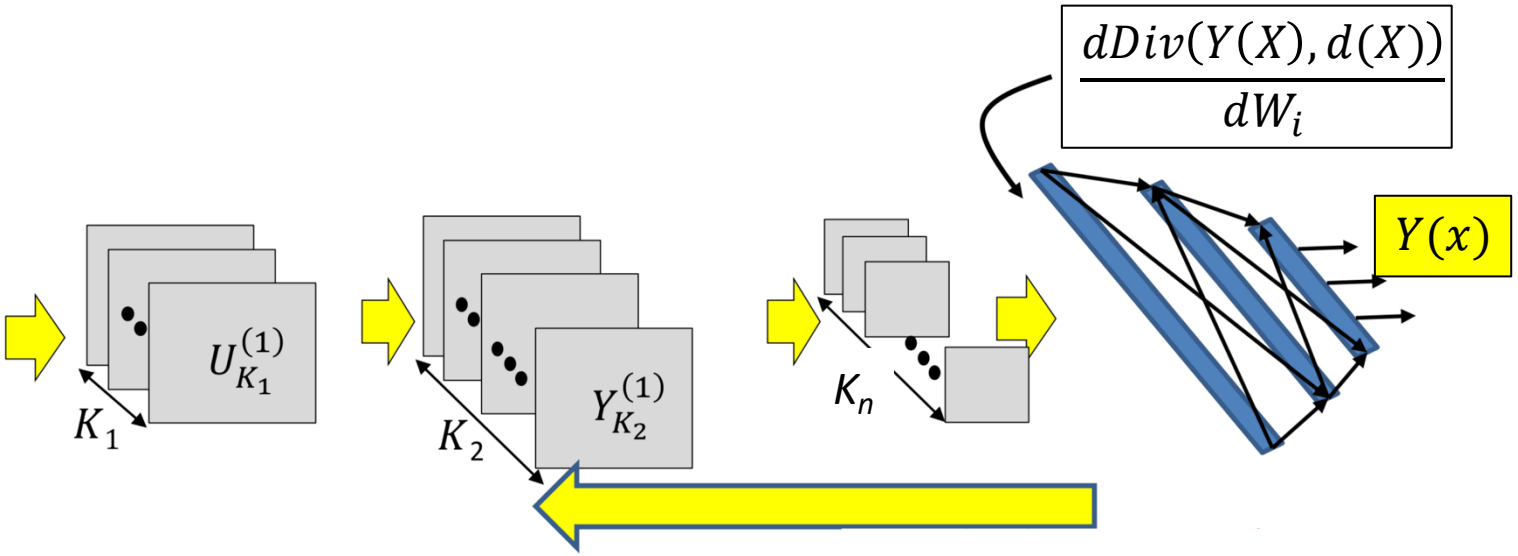
- El entrenamiento de una CNN es similar al de las redes neuronales, utilizando Backpropagation y gradiente descendiente.

Redes CNN: Aprendizaje



- Forward propagation
  - Como antes, en este paso se calcula la salida de la red y se compara con la etiqueta deseada.
  - Recordar que la salida se evalúa a través de una función de error, fuera de la red.

Redes CNN: Aprendizaje



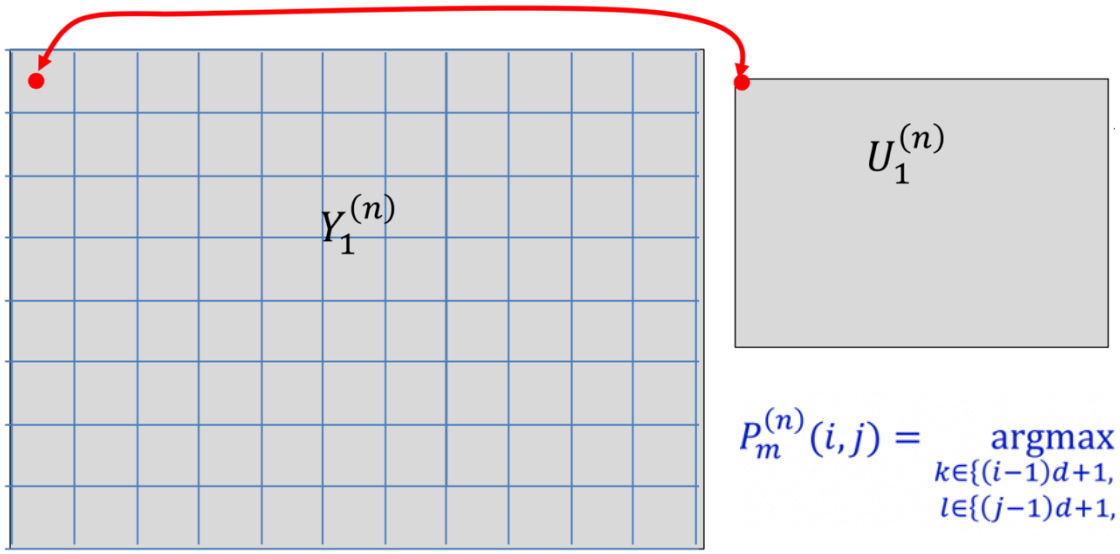
El ajuste de estos pesos, require otras consideraciones

- Backpropagation
  - Backprop desde la capa MLP a las capas Conv, requiere considerar:
  - Las capas pooling
  - Los parámetros compartidos en las capas convolucionales

Redes CNN: Aprendizaje

- Propagación hacia atrás en la capa pooling
  - En la capa Pooling, no tenemos ningún parámetro que actualizar; por lo tanto, lo único que debemos hacer es distribuir los gradientes donde correspondan; esto es:
  - En el paso forward; por ejemplo, con max pooling, seleccionamos el valor máximo de cada región y los transferimos a la siguiente capa. Por lo tanto, en la propagación hacia atrás, el gradiente no debe afectar a los elementos de la matriz que no se incluyeron en el paso hacia delante
  - Para actualizar sólo aquellos parámetros que contribuyeron en el paso hacia delante, se utiliza una máscara que recuerda la posición de los valores utilizados en el paso forward, de manera que luego podemos utilizarlos para transferir los gradientes, así:

Redes neuronales convolucionales



$\cdots\cdots$ $Div(Y(X), d(X))$

$$P_m^{(n)}(i,j) = \underset{\substack{k\in\{(i-1)d+1,\, id\},\\ l\in\{(j-1)d+1,\, jd\}}}{\operatorname{argmax}} Y_m^{(n)}(k,l)$$

$$\frac{dDiv(Y(X), d(X))}{dY_m^{(n)}(k,l)} = \begin{cases} \dfrac{dDiv(Y(X), d(X))}{dU_m^{(n)}(i,j)} & Si\ Y_m^{(n)}(k,l) = P_m^{(n)}(i,j) \\ 0 & de\ lo\ contrario \end{cases}$$

- Las derivadas c.r.a los términos Y que no contribuyen en el mapa Max son 0